

Второй этап группового проекта: Алгоритмы и программная реализация Неравновесная агрегация и фрактальные структуры (модель DLA)

Казазаев Даниил Михайлович Симонова Виктория Игоревна
Малюга Валерия Васильевна Ларина Наталья Денисовна
Лисенков Егор Романович

2026-04-11

Содержание

1	Введение	2
2	Базовый алгоритм DLA на решётке	2
2.1	Инициализация	2
2.2	Основной цикл роста	3
2.3	Ускорение проверки прилипания	4
3	Алгоритмы расчёта фрактальной размерности	4
3.1	Метод радиуса гирации	4
3.2	Метод подсчёта ячеек (box-counting)	4
4	Программная реализация	5
4.1	Выбор языка и инструментов	5
4.2	Структуры данных	5
4.3	Псевдокод базового шага DLA	6
5	Модификации алгоритма	7
5.1	Химически ограниченная агрегация (RLA)	7
5.2	Баллистическая агрегация	7
5.3	Кластер-кластерная агрегация (DLCA)	7
6	Вычислительные аспекты и оптимизация	7
6.1	Ограничение области блуждания	7
6.2	Использование списка граничных узлов	7
6.3	Динамическое выделение памяти	8
7	Заключение	8

Список иллюстраций

Список таблиц

1 Введение

В рамках первого этапа проекта были рассмотрены теоретические основы неравновесной агрегации и фрактальных структур на примере модели DLA (Diffusion-Limited Aggregation). Настоящий документ посвящён **второму этапу** - детальному описанию алгоритмов и подходов к программной реализации моделирования роста фрактальных кластеров.

Цель данного этапа - формализовать вычислительные процедуры, необходимые для воспроизведения модели DLA и её модификаций, а также для расчёта фрактальных характеристик получаемых структур. Основное внимание уделено эффективности алгоритмов, структурам данных и методам оптимизации вычислений.

2 Базовый алгоритм DLA на решётке

Модель DLA оперирует дискретным пространством - квадратной решёткой. Состояние каждого узла характеризуется признаком: свободен (0) или занят частицей кластера (1). Алгоритм роста кластера включает следующие шаги.

2.1 Инициализация

1. Задать размер расчётной области $N \times N$ узлов. Для удобства индексации можно использовать массив `grid[1..N, 1..N]`.
2. Поместить затравочную частицу в центр области: `grid[N div 2 + 1, N div 2 + 1] := 1`.
3. Инициализировать переменную текущего максимального радиуса кластера $R_{\max} = 0$ (в единицах шага решётки).
4. Установить параметры:
 - R_{spawn} - радиус окружности, на которой генерируются новые частицы (например, $R_{\text{spawn}} = R_{\max} + 2$).

- R_{kill} - радиус обрыва блуждания (например, $R_{\text{kill}} = \max(3 \cdot R_{\text{max}}, R_{\text{max}} + 10)$).

2.2 Основной цикл роста

Повторять до достижения заданного числа частиц $N_{\text{particles}}$ или до заполнения области:

2.2.1 1. Генерация новой частицы

- Выбрать случайный угол $\alpha \in [0, 2\pi)$ с помощью генератора равномерного распределения.
- Вычислить координаты старта:

$$x_{\text{start}} = x_{\text{center}} + \text{round}(R_{\text{spawn}} \cdot \cos \alpha), \quad y_{\text{start}} = y_{\text{center}} + \text{round}(R_{\text{spawn}} \cdot \sin \alpha).$$

- Если узел $(x_{\text{start}}, y_{\text{start}})$ уже занят, повторить генерацию угла.

2.2.2 2. Случайное блуждание

- Пока частица не прилипла и не достигла R_{kill} :
 - Выбрать случайное направление из четырёх возможных: влево, вправо, вверх, вниз. Вероятность каждого $1/4$.
 - Вычислить новые координаты (x', y') .
 - Если (x', y') выходит за границы расчётной области, частица отражается (либо уничтожается, в зависимости от реализации). В простейшем случае при выходе за R_{kill} частица уничтожается, и запускается новая.
 - Проверить **условие прилипания**: если хотя бы один из четырёх соседних узлов занят, частица прилипает к текущему узлу.
 - * Установить $\text{grid}[x', y'] := 1$.
 - * Обновить $R_{\text{max}} = \max(R_{\text{max}}, \sqrt{(x' - x_c)^2 + (y' - y_c)^2})$.
 - * Увеличить счётчик частиц.
 - * Перейти к генерации следующей частицы.

2.2.3 3. Адаптация параметров

После добавления каждой частицы (или через определённое число шагов) пересчитываются: - R_{max} (максимальное расстояние от центра до

любой точки кластера). - $R_{\text{spawn}} = R_{\text{max}} + 2$. - $R_{\text{kill}} = \max(3R_{\text{max}}, R_{\text{max}} + 10)$.

2.3 Ускорение проверки прилипания

Проверка всех четырёх соседей на каждом шаге блуждания может быть оптимизирована с использованием **списка граничных узлов**. Граничными называются свободные узлы, соседние хотя бы с одним занятым. При добавлении новой частицы в кластер обновляется этот список: новый узел удаляется из списка, а его свободные соседи добавляются (если их там ещё нет). Тогда условие прилипания сводится к проверке принадлежности текущего положения частицы к множеству граничных узлов. В двумерном случае это даёт ускорение до ~4 раз.

3 Алгоритмы расчёта фрактальной размерности

3.1 Метод радиуса гирации

Радиус гирации характеризует среднеквадратичное расстояние частиц кластера от его центра масс. Для его вычисления необходимо хранить координаты всех частиц, входящих в кластер.

Алгоритм: 1. После добавления N частиц (или периодически во время роста) вычислить координаты центра масс:

$$\mathbf{R}_{cm} = \frac{1}{N} \sum_{i=1}^N \mathbf{r}_i.$$

2. Вычислить квадрат радиуса гирации:

$$R_g^2 = \frac{1}{N} \sum_{i=1}^N |\mathbf{r}_i - \mathbf{R}_{cm}|^2.$$

3. Сохранить пару $(\ln N, \ln R_g)$ для последующего линейного регрессионного анализа. Наклон прямой $\ln N = D \ln R_g + C$ даёт оценку фрактальной размерности D .

3.2 Метод подсчёта ячеек (box-counting)

Этот метод применяется к уже сформированному кластеру и не требует информации о порядке роста.

Алгоритм: 1. Определить минимальную охватывающую рамку кластера: $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$. 2. Для серии размеров ячейки $\epsilon = 2, 4, 8, \dots, 2^k$, где $2^k \leq \max(x_{\max} - x_{\min}, y_{\max} - y_{\min})$: - Разбить рамку на ячейки размером $\epsilon \times \epsilon$. - Подсчитать количество ячеек $N(\epsilon)$, содержащих хотя бы одну частицу кластера. 3. Построить график зависимости $\ln N(\epsilon)$ от $\ln(1/\epsilon)$. Наклон аппроксимирующей прямой есть фрактальная размерность D .

4 Программная реализация

4.1 Выбор языка и инструментов

В рамках учебного курса предполагается использование языка **Pascal** (среда Free Pascal или Delphi) с графическим пакетом MPP (Modeling of Physical Processes), описанным в пособии [1]. Однако для независимой разработки проект также может быть реализован на **Python** с библиотеками NumPy и Matplotlib, либо на **C++**.

Основные компоненты программы:

1. **Модуль инициализации** - задание параметров модели (размер сетки, число частиц, вероятности прилипания и т.д.).
2. **Модуль DLA-симуляции** - реализация базового алгоритма с возможностью выбора модификаций (RLA, баллистическая агрегация, кластер-кластерная).
3. **Модуль визуализации** - отображение растущего кластера в реальном времени.
4. **Модуль анализа** - вычисление фрактальной размерности, построение графиков.
5. **Модуль ввода/вывода** - сохранение координат частиц, логов, экспорт изображений.

4.2 Структуры данных

- **Решётка:** двумерный массив `grid` типа `array[1..N, 1..N] of Byte`. Значение 0 - свободно, 1 - занято.
- **Список частиц кластера:** динамический массив координат для расчёта R_g .
- **Список граничных узлов:** для ускорения проверки прилипания можно использовать множество (хеш-таблицу) или массив логических

флагов.

4.3 Псевдокод базового шага DLA

```
procedure GrowDLA;
var
x, y, nx, ny: integer;
dir: integer;
stuck: boolean;
begin
x := RandomSpawnX; y := RandomSpawnY;
while true do
begin
dir := Random(4);
case dir of
0: nx := x - 1; ny := y;
1: nx := x + 1; ny := y;
2: nx := x; ny := y - 1;
3: nx := x; ny := y + 1;
end;
if DistanceToCenter(nx, ny) > RKill then
break;
stuck := false;
if IsNeighborOccupied(nx, ny) then
begin
AddParticle(nx, ny);
UpdateRMax;
stuck := true;
break;
end;
x := nx; y := ny;
end;
if not stuck then
end;
```

5 Модификации алгоритма

5.1 Химически ограниченная агрегация (RLA)

Вводится параметр $p \in (0, 1]$ - вероятность прилипания. При проверке соседей, если обнаружен занятый узел, дополнительно генерируется случайное число $r \in [0, 1)$. Прилипание происходит только если $r < p$. В противном случае частица продолжает блуждание. Это позволяет частицам проникать вглубь фьордов, увеличивая плотность кластера.

5.2 Баллистическая агрегация

Частицы движутся не случайно, а по прямым траекториям из точки старта. Алгоритм: - Выбрать направление вектора скорости (угол α). - Вычислить траекторию до пересечения с существующим кластером (или до выхода за границу). - Прилипание происходит в первой точке контакта.

Для эффективного поиска пересечения можно использовать алгоритм Брезенхема для растеризации отрезка.

5.3 Кластер-кластерная агрегация (DLCA)

В начальный момент все частицы размещены случайно в расчётной области и совершают случайные блуждания. При столкновении двух кластеров они сливаются в один. Алгоритм требует отслеживания связности частиц и пересчёта положения центра масс объединённого кластера.

6 Вычислительные аспекты и оптимизация

6.1 Ограничение области блуждания

Без ограничений частица, ушедшая далеко от кластера, может блуждать очень долго, прежде чем вернётся. Введение R_{kill} и повторный запуск с R_{spawn} существенно ускоряет симуляцию.

6.2 Использование списка граничных узлов

Поддержание актуального множества свободных узлов, соседних с кластером, позволяет проверять прилипание за $O(1)$ вместо $O(4)$ проверок соседей. При большом размере кластера выигрыш значителен.

6.3 Динамическое выделение памяти

Поскольку число частиц может достигать десятков и сотен тысяч, хранение всех координат в массиве фиксированного размера неэффективно. Рекомендуется использовать динамические массивы (например, `TList` в Pascal) или стандартные контейнеры в других языках.

7 Заключение

На втором этапе проекта разработаны детальные алгоритмы моделирования неравновесной агрегации по схеме DLA и её модификаций. Определены ключевые структуры данных, предложены методы оптимизации для ускорения вычислений. Описаны алгоритмы расчёта фрактальной размерности - радиуса гирации и box-counting. Данная алгоритмическая база является основой для непосредственной программной реализации проекта на следующем этапе.

8 Литература

- [1] Медведев Д. А., Куперштох А. Л., Прууэл Э. Р., Сатонкина Н. П., Карпов Д. И. Моделирование физических процессов и явлений на ПК: Учеб. пособие. Новосибирск: НГУ, 2010. 101 с.
- [2] Witten T. A., Sander L. M. Diffusion-Limited Aggregation, a Kinetic Critical Phenomenon // Physical Review Letters. 1981. V. 47, N 19. P. 1400–1403.
- [3] Федер Е. Фракталы. М.: Мир, 1991.